

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 10%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I Arava Raja(192365033) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Arava Raja

Signature of the student:

Annexure A

Descriptive Written Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.
2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Question 1

Objective

To understand the different types of intelligent agents and analyze how they can be applied in a smart home automation system for comfort, energy efficiency, and security.

Problem

A smart home automation system controls lighting, temperature, and security using real-time inputs such as motion detection, temperature, and time of day. The system also learns user preferences over time. Explain the different intelligent agents and determine the most suitable approach.

Solution

1. Simple Reflex Agent

A Simple Reflex Agent makes decisions based only on the current input without considering previous states.

Working in Smart Home

- Turns ON lights when motion is detected.
- Turns OFF lights when no movement is detected.
- Activates the alarm when an intruder is detected.

Advantages

- Fast response
- Easy to implement

Disadvantages

- Cannot remember previous events.
- Cannot learn user preferences.

2. Model-Based Reflex Agent

A Model-Based Agent keeps track of the environment using an internal model.

Working in Smart Home

- Remembers whether lights are already ON or OFF.
- Tracks room occupancy.
- Maintains current temperature information.

Advantages

- Better decision making.
- Works in partially observable environments.
-

Disadvantages

- More complex.
- Requires additional memory.

3. Goal-Based Agent

A Goal-Based Agent selects actions that help achieve a specific goal.

Working in Smart Home

Goals include:

- Maintaining comfortable room temperature.
- Saving electricity.
- Keeping the house secure.

Example:

If temperature increases, the agent decides whether to start the air conditioner to reach the desired temperature.

Advantages

- Makes intelligent decisions.
- Flexible for different situations.

Disadvantages

- Requires search algorithms.
- Takes more computation time.

4. Utility-Based Agent

A Utility-Based Agent chooses the action that provides the highest overall benefit.

Working in Smart Home

The agent balances:

- Comfort
- Energy consumption
- Security

Example:

Instead of always turning ON the AC, it compares:

- Electricity cost
- User comfort
- Outdoor temperature

and selects the best option.

Advantages

- Produces optimal decisions.
- Balances multiple objectives.

Disadvantages

- Utility function is difficult to design.

Comparison of Intelligent Agents

Agent Type	Memory	Goal	Learning	Best Use
Simple Reflex	No	No	No	Basic lighting
Model-Based	Yes	No	Limited	Security monitoring
Goal-Based	Yes	Yes	No	Temperature control
Utility-Based	Yes	Yes	Yes	Overall smart home management

Most Suitable Approach

A **Hybrid Approach** combining:

- Model-Based Agent
- Goal-Based Agent
- Utility-Based Agent

is the most effective.

Reasons

- Stores previous information.
- Learns user preferences.
- Minimizes electricity usage.
- Maximizes comfort.
- Provides better security.
- Makes optimal decisions under different situations.

Python code:

```
motion = True
temperature = 31
time = "Night"
door_open = True

print("SMART HOME AUTOMATION")
print("-----")

# Lighting
if motion:
    print("Lights : ON")
else:
    print("Lights : OFF")

# Temperature Control
if temperature >= 30:
    print("Air Conditioner : ON")
else:
    print("Air Conditioner : OFF")

# Security
if time == "Night" and door_open:
    print("Security Alert : Door Open!")
else:
    print("Security Status : Safe")

print("\nAgent Decision Completed Successfully.")
```

Console



▶ Run

```
Run started
Initializing environment
Installing packages
Running code
SMART HOME AUTOMATION
-----
Lights : ON
Air Conditioner : ON
Security Alert : Door Open!

Agent Decision Completed Successfully.
Run completed in 2017.5ms
```

Discussion

A smart home requires more than simple reactions. It should remember previous states, achieve user goals, and maximize comfort while reducing energy consumption. Therefore, combining multiple intelligent agent types provides a more reliable and adaptive solution.

Conclusion

A hybrid intelligent agent offers the best performance because it combines memory, goal-oriented behavior, and utility optimization, ensuring comfort, energy efficiency, and security.

Question 2

Objective

To explain how Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) solve an unknown maze and compare their performance.

Problem

A robot must find the exit of an unknown maze using uninformed search techniques. Compare Uniform Cost Search (UCS) and Iterative Deepening Search (IDS), discuss their complexities, relate them to intelligent agents, and recommend the best combination.

Solution

Uniform Cost Search (UCS)

Definition

Uniform Cost Search expands the node with the smallest cumulative path cost.

Working

1. Start from the initial position.
2. Store paths in a priority queue.
3. Expand the lowest-cost node.
4. Continue until the goal is reached.
- 5.

Advantages

- Finds the least-cost path.
- Works with varying movement costs.
- Complete and optimal.

Disadvantages

- Requires large memory.
- Can be slow for large search spaces.

Time Complexity

$$O(b^{(1 + C/\epsilon)})^*$$

where:

- b = branching factor
- C^* = optimal path cost
- ϵ = minimum edge cost

Space Complexity

$$O(b^{(1 + C/\epsilon)})^*$$

Iterative Deepening Search (IDS)

Definition

IDS repeatedly performs Depth-Limited Search with increasing depth limits.

Working

Depth = 0

↓

Depth = 1

↓

Depth = 2

↓

Continue until goal is found.

Advantages

- Very low memory usage.
- Complete.
- Suitable when search depth is unknown.

Disadvantages

- Repeats node expansion.
- Slightly slower because of repeated searches.

Time Complexity

$$O(b^d)$$

Space Complexity

$$O(b \times d)$$

Comparison

Feature	UCS	IDS
Complete	Yes	Yes
Optimal	Yes	Only for equal costs
Memory Usage	High	Low
Speed	Slower	Faster for shallow goals
Handles Different Costs	Yes	No

Relation to Intelligent Agents

Simple Reflex Agent

- Cannot solve unknown maze.
- Reacts only to current state.

Model-Based Agent

- Builds an internal map.
- Remembers explored locations.

Goal-Based Agent

- Searches for the exit.
- Chooses actions leading toward the goal.

Utility-Based Agent

- Selects paths with minimum cost.
- Avoids risky or expensive routes.

Best Combination

The most suitable combination is:

Goal-Based Agent + Uniform Cost Search Justification

- Goal-Based Agent focuses on reaching the exit.
- UCS guarantees the least-cost path.
- Works effectively when movement costs differ.
- Produces optimal solutions.

If all movement costs are equal and memory is limited:

Goal-Based Agent + IDS

is preferred because IDS requires significantly less memory.

Pythion code:

```
import heapq

graph = {
    'S': [('A', 2), ('B', 4)],
    'A': [('C', 3)],
    'B': [('D', 2)],
    'C': [('G', 4)],
    'D': [('G', 1)],
    'G': []
}

def ucs(start, goal):
    pq = [(0, start, [])]
    visited = set()

    while pq:
        cost, node, path = heapq.heappop(pq)

        if node in visited:
            continue

        visited.add(node)
        path = path + [node]

        if node == goal:
            return path, cost

        for neighbour, weight in graph[node]:
            heapq.heappush(pq, (cost + weight, neighbour, path))

path, cost = ucs('S', 'G')

print("Uniform Cost Search")
print("Path :", " -> ".join(path))
print("Cost :", cost)
```

```
Console 🗑️ 📄 ▶ Run
```

```
Run started
Initializing environment
Installing packages
Running code
Uniform Cost Search
Path : S -> B -> D -> G
Cost : 7
Run completed in 3622.199999999255ms
```

Discussion

In an unknown maze, the robot needs to explore efficiently while making intelligent decisions. UCS is ideal when path costs vary because it always finds the optimal path. IDS is a good alternative when memory is limited and all moves have equal cost. Choosing the appropriate agent design further improves navigation and decision-making.

Conclusion

For unknown maze navigation, **Uniform Cost Search with a Goal-Based Agent** is the most effective solution because it guarantees an optimal path while enabling intelligent decision-making. When memory is a major constraint and movement costs are equal, **Iterative Deepening Search** becomes a practical alternative.